

Name

keyboard – Activates international console keyboards.

Syntax

keyboard *country*

Description

The *keyboard* utility lets you choose the character set for XENIX to use with the console keyboard. You may choose any of the following as the *country* :

FR	French Keyboard
GR	German Keyboard
IT	Italian Keyboard
SP	Spanish Keyboard
UK	UK, British Keyboard
US	US Keyboard, resident when XENIX is loaded

The US keyboard table is resident when XENIX boots. If you want a different keyboard table to automatically load when XENIX boots, place the following line at the end of the */etc/rc* file:

```
/bin/keyboard XX < /dev/console > /dev/console 2>&1
```

where XX is the two letter abbreviation for the desired country as described above.

XENIX treats characters that appear on an international (non-US) keyboard and also appear on a US keyboard exactly as if they had been typed on a US keyboard. This happens even if the position of the key is different. However, XENIX sets the 8th bit on characters that do not appear on US keyboards but do appear on international keyboards (for example, the n-tilde on the Spanish keyboard). XENIX assumes, in most cases, that characters are 7 bits and does not handle these "high-bit" characters properly. For example, n-tilde and other high-bit characters cannot be used as a character in a filename and may cause unexpected behavior in other cases. For this reason, high-bit characters should be avoided if at all possible.

However, high-bit characters can be put into a file just like normal ASCII characters. To use the high-bit characters, you must enter the following command:

```
stty -istrip
```

Normally, XENIX strips the high bit from incoming characters because it expects 7 bit characters. If you plan to use the high-bit characters regularly, you should put the *stty -istrip* command in your *profile* file (if you use */bin/sh* as your shell) or in your *.login* file (if you use */bin/csh* as your shell). Unfortunately, the text editors always strip input characters to 7 bits or refuse to work when a file contains high-bit characters. To place high-bit characters in a file, use the *cat* utility.

Dead Key Processing

There are certain keys on the French, German, and Spanish keyboards that do not produce a keystroke when pressed alone. These keys are called "dead keys". The German keyboard uses the key with ' and ' as a dead key, the French keyboard uses the key with ^ and the diaeresis, and the Spanish keyboard has two dead keys: one with ' and diaeresis on it and the other with ' and ^ on it.

When a dead key is pressed alone, a keystroke is not registered until another key is pressed. If the next key pressed can be legally combined with the dead key, the combination is registered as a single character and put in the keyboard queue. For example, if the ^ key is pressed on the French keyboard and followed by pressing the 'a' key, a single character is put in the keyboard queue whose ASCII code is hexadecimal 83 (a high-bit character). This is echoed on the screen as an 'a' with a circumflex on top (if *-istrip* was set as described above).

If the next key pressed cannot be legally combined with the dead key, the two characters that represent the dead key and the following key are all put in the keyboard queue. For example, if the ^ key is pressed on the French keyboard and followed by pressing the 'x' key, two characters are put in the keyboard queue, namely, the ^ character and the 'x' character. If a dead key is followed by a space, only the dead key is put in the keyboard queue. For example, if the ^ key is pressed on the French keyboard and followed by pressing the space bar, only the ^ character is put in the keyboard queue.

Displaced Keys

All keys that are present on the US keyboard are present on all international keyboards. To make room for the high-bit international characters, some keys had to be displaced. For example, to make room for the British pound sign on the UK keyboard, the grave accent (') was displaced. In order to read a grave accent from the UK keyboard, press the acute accent key (') while the CTRL and ALT keys are pressed simultaneously. The acute accent key (') is marked on the UK keyboard with a grave accent (') on the front face of the key. This indicates that CTRL-ALT-acute accent produces a grave accent character. The same is true for all keys with a character printed on the front face of the key. In order to read the character from the keyboard, press the key while holding down both the CTRL and ALT keys.

Notes

Stand-alone programs such as **boot** and **badtrack** use the US keyboard table.

The "section" key (which shares the '6' key on the French keyboard and the '3' key on the German keyboard) cannot be printed properly in XENIX because its ASCII value is the same as CTRL-U.

Name

kill – Terminates a process.

Syntax

kill [*-signo*] *processid* ...

Description

Kill sends signal 15 (terminate) to the specified processes. This will normally kill processes that do not catch or ignore the signal. The process number of each asynchronous process started with & is reported by the shell (unless more than one process is started in a pipeline, in which case the number of the last process in the pipeline is reported). Process numbers can also be found by using *ps*(C).

For example, if process number 0 is specified, all processes in the process group are signaled.

The killed process must belong to the current user unless he is the super-user.

If a signal number preceded by – is given as the first argument, that signal is sent instead of the terminate signal (see *signal*(S)). In particular, “kill –9 ...” is a sure kill.

See Also

kill(S), ps(C), sh(C), signal(S)



Name

line – Reads one line.

 Syntax

line

Description

Line copies one line (up to a NEWLINE) from the standard input and writes it on the standard output. It returns an exit code of 1 on end-of-file and always prints at least a NEWLINE. It is often used within shell files to read from the user's terminal.

See Also

gets(CP), sh(C)



Name

ln – Makes a link to a file.

Syntax

ln *name1 name2*

ln *name1 ... directory*

Description

A link is a directory entry referring to a file; the same file (together with its size, all its protection information, etc.) may have several links to it. There is no way to distinguish a link to a file from its original directory entry. Any changes to the file are effective independent of the name by which the file is known.

In the first case, *ln* creates a link to the existing file, *name1*. The *name2* argument is a new name referring to the same file contents as *name1*.

In the second case, *directory* is the location of a directory into which one or more links are created with corresponding file names.

It is forbidden to link to a directory or to link across file systems.

See Also

cp(C), **mv(C)**, **rm(C)**



Name

local, lp - Allows printing on terminal printer.
 spool, lp - Allows printing on system printer.

Syntax

lp [*options*] *filename*
 local lp [*options*] *filename*
 spool lp [*options*] *filename*

Description

If you have a printer connected to your terminal, you can print on it, or you can print on the system printer. Refer to the **lp (C)** command in the *Reference Volume I* manual for available options.

If you are using a printer that can use non-interpretive mode, set the FORMS variable to "x". Non-interpretive mode means that your printer's dip switches are set to CR = CR, LF = NL, and your printer can do formfeeds and tabs. To set the FORMS variable, type:

```
FORMS=x;export FORMS
```

If you can't set your printer to use non-interpretive mode, set your printer to CR = NL (or CR = CR LF) and use interpretive mode. To do so, set the FORMFEED variable to "no" by typing:

```
FORMFEED=no; export FORMFEED
```

Note that if you are using **csh**, you must use the **setenv** command to set variables. For example, to set the FORMFEED variable, type:

```
setenv FORMFEED no
```

Using Your Local Printer

The local printer is the one connected to your terminal.

If you are using a DT-1, it must be modified before you can use it for local printing. Consult your Radio Shack Computer Center representative about this modification.

If you are using a DT-100, to do local printing you need a Tandy DT-100 Terminal Interface (Cat. No. 26-1198).

After you connect your printer; turn it on, make sure that it is online, and set the printer for either non-interpretive or interpretive mode (as shown earlier). Now type:

```
local lp filename
```

Press RETURN.

If the printer does not start or if it stops during printing, check the printer before pressing any key. If the printer goes offline for more than 10 minutes, the printer times out and cancels the print job. Correct the problem that caused the printer to go offline, place the printer back online, and restart the print job.

Setting Form Size in Interpretive Mode

In interpretive mode, XENIX automatically sets the size of the form on which you print to 66 lines (6 lines = 1 inch). (Non-interpretive mode issues a FF code to the printer.) This does not mean that you actually print 66 lines of text on the form. The printer prints whatever you have specified and then goes to the top of the next form. From the top of the form to the bottom is 66 lines.

If you are using interpretive mode, you can change the number of lines per form by using the following commands:

```
local lp -F "p=x" filename
```

-F tells XENIX that you want to change the size of the form. For x, substitute the number of lines you want per form. For example to print checks or invoices that are 30 line (5 inches) from top to bottom, type:

```
local lp -F "p=30" filename
```

Notes

While doing local printing, do not press any key on the keyboard unless your terminal screen displays a message asking you to do so. Do not attempt to terminate the printing process by pressing BREAK or any other key. Inadvertently pressing a key may cause extra characters to be printed on the printer and may cause the printer, the terminal, or both to end processing.

Using -F with lp is not a way to paginate. You are not specifying the number of text lines on the page, but rather the number of lines from the top to the bottom of a form. Do not attempt to use -F as a paginating device. It will not work as such.

If you wish to be able to print locally all the time, set the variable LPR to "local." For example, if you are using the Bourne Shell, you can add the following line to your *profile* file:

```
LPR=local;export LPR
```

Now, when you log in, you are set to do local printing and need not use the local command to do so. Using lp means printing on the local printer. The variable LPR is set to "spooler" by default. If you have set LPR to "local," and wish to print something on the system printer, you may set LPR to "spooler", or use:

```
spool lp filename
```

to print files on the system printer for this command only.

By default, local printing always sends a formfeed character at the end of every print job. You can turn off this option by setting the NOFF (No Form Feed) environment variable to “true”:

```
NOFF=true; export NOFF
```

To return to the default state, set NOFF equal to “false.”

See Also

default(M), lp(C)



Name

`login` – Gives access to the system.

Description

The *login* command is used at the beginning of each terminal session and allows you to identify yourself to the system. It cannot be invoked except when a connection is first established, or after the previous user has logged out by sending an end-of-file (CTRL-D) to his initial shell.

Login asks for your user name, and if appropriate, your password. Echoing is turned off (where possible) during the typing of your password, so it will not appear on the written record of the session.

At some installations, an option may be invoked that will require you to enter a second “external” password. This will occur only for dial-up connections, and will be prompted by the message “External security:”. Both passwords are required for a successful login.

If password aging has been invoked by the super-user on your behalf, your password may have expired. In this case, you will be shunted into *passwd*(C) to change it, after which you may attempt to log in again.

If you do not complete the login successfully within a certain period of time (e.g., one minute), you are likely to be returned to the “login:” prompt or silently disconnected from a dial-up line.

After a successful login, accounting files are updated, you are informed of the existence of any mail, and the start-up profile files (i.e., */etc/profile* and *\$HOME/.profile*) (if any) are executed (see *profile*(M)). *Login* initializes the user and group IDs and the working directory, then executes a command interpreter (usually *sh*(C)) according to specifications found in the */etc/passwd* file. Argument 0 of the command interpreter is a dash (–) followed by the last component of the interpreter’s pathname. The *environment* (see *environ*(M)) is initialized to:

`HOME= your-login-directory`

`PATH=:/bin:/usr/bin`

Initially, *umask* is set to octal 022 by *login*.

Files

/etc/utmp

Accounting file

/etc/wtmp	Accounting file
/usr/spool/mail/ <i>your-name</i>	Mailbox for user <i>your-name</i>
/etc/motd	Message of the day
/etc/passwd	Password file
/etc/profile	System profile
\$HOME/.profile	Personal profile
/etc/default/login	Default login file

Notes

File size limitation is set by initializing the ULIMIT variable to the number of 1K-byte blocks. .SH "See Also" default(M), environ(M), getty(M), mail(C), newgrp(C), passwd(C), passwd(M), profile(M), sh(C), su(C), umask(C)

Diagnostics

Login incorrect

The user name or the password is incorrect.

No shell, cannot open password file, no directory:

Your account has not been set up properly.

Your password has expired. Choose a new one.

Password aging is in effect and yours has expired.

Name

logname – Gets login name.

Syntax

logname

Description

Logname returns the contents of the environment variable, \$LOGNAME, which is set when a user logs into the system.

See Also

env(C), environ(M), login(M), logname(S)



Name

look – Finds lines in a sorted list.



Syntax

look [**-df**] *string* [*file*]

Description

Look consults a sorted *file* and prints all lines that begin with *string*. It uses binary search.

The **-d** and **-f** options affect comparisons as in *sort*(C):

-d Dictionary order: Only letters, digits, tabs, and spaces are compared.

-f Fold: Uppercase letters compare equally to lowercase letters.

If no *file* is specified, */usr/dict/words* is assumed with the collating sequence, **-df**.



Files

/usr/dict/words

See Also

grep(C), sort(C)



Name

`lp`, `lp̄`, `cancel` – Send/cancel requests to lineprinter.

Syntax

```
lp [ options... ][ name... ]
lp̄ [ options... ][ name... ]
cancel [ request IDs ][ printers ]
```

Description

Lp causes the named files and associated information (collectively called a “request”) to be printed by a lineprinter. *Lp* and *lp̄* are equivalent commands and may be used interchangeably. If no filenames are mentioned, the standard input is assumed. The filename, `–`, stands for the standard input and may be supplied on the command line in conjunction with named *files*. The order in which *files* appear is the order in which they will be printed.

Lp associates a unique request with each request and prints it on the standard output. This request can be used later to cancel (see *cancel*) or find the status of the request (see *lpstat(C)*).

The following options to *lp* may appear in any order and may be intermixed with file names:

- c** Makes copies of the *files* to be printed immediately when *lp* is invoked. Normally, *files* will not be copied, but will be linked whenever possible. If the `–c` option is not given, then the user should be careful not to remove any of the *files* before the request has been printed in its entirety; any changes made to the named *files* after the request is made but before it is printed will be reflected in the printed output.
- ddest** Chooses *dest* as the printer or class of printers to do the printing. If *dest* is a printer, then the request will be printed only on that specific printer. If *dest* is a class of printers, then the request will be printed on the first available printer that is a member of the class. Under certain conditions (for example, printer unavailability or file space limitation), requests for specific destinations may not be accepted (see *accept(M)* and *lpstat(C)*). By default, *dest* is taken from the environment variable, `LPDEST` (if it is set). Otherwise, a default destination (if one exists) for the computer system is used. Destination names vary between systems (see *lpstat(C)*).
- m** Sends mail (see *mail(C)*) after the files have been printed. By default, no mail is sent upon normal completion of the print request.

- `-nnumber` Prints *number* of copies of the output. The default is one.
- `-ooption` Specifies printer-dependent or class-dependent *options*. Several such *options* may be collected by specifying the `-o` keyletter more than once. For more information about what is valid for *options*, see `lpadmin(C)`.
- `-r` Removes file after sending it.
- `-s` Suppresses messages from `lp(C)` such as “request id is ...”.
- `-ttitle` Prints *title* on the banner page of the output. If *title* is more than one word, you must enclose it within quotes.
- `-w` Writes a message on the user’s terminal after the *files* have been printed. If the user is not logged in, then mail will be sent instead.

The `/etc/default/lpd` file contains the setting of the BANNERS variable, whose value is the number of pages printed as a banner identifying each printout. This is normally set to either 1 or 2.

Cancel cancels lineprinter requests that were made by the `lp(C)` command. The command-line arguments may be either requests (as returned by `lp(C)`) or *printer* names (for a complete list, use `lpstat(C)`). Specifying a request cancels the associated request even if it is currently printing. Specifying a *printer* cancels the request which is currently printing on that printer. In either case, the cancellation of a request that is currently printing frees the printer for its next available request. The user’s identification and accounting data pool area contains BANNERS setting.

Files

`/etc/passwd`
`/usr/spool/lp/*`
`/etc/default/lpd`

Notes

The *cancel* command does not generate a top of form to the printer when it cancels the job.

See Also

`accept(C)`, `default(M)`, `enable(C)`, `lpadmin(C)`, `lpsched(C)`, `lpstat(C)`, `mail(C)`

Name

`lpadmin` – Configures the lineprinter spooling system.

Syntax

```
/usr/lib/lpadmin -p printer [ options... ]
/usr/lib/lpadmin -x dest
/usr/lib/lpadmin -d [ dest ]
```

Description

Lpadmin configures the lineprinter spooling system to describe printers, classes, and devices. It is used to add and remove destinations, change membership in classes, change devices for printers, change printer interface programs, and to change the system default destination. System managers may also use *lprint(C)* to add new printing destinations to the system. *Lpadmin* may not be used when the lineprinter scheduler, *lpsched(C)*, is running, except where noted below.

Exactly one of the `-p`, `-d`, or `-x` options must be present for every legal invocation of *lpadmin*.

- `-d[dest]` Makes *dest*, an existing destination, the new system default destination. If *dest* is not supplied, then there is no system default destination. This option may be used when *lpsched(C)* is running. No other *options* are allowed with `-d`.
- `-xdest` Removes destination, *dest*, from the LP system. If *dest* is a printer and is the only member of a class, then the class will be deleted, too. No other *options* are allowed with `-x`.
- `-pprinter` Names a *printer* to which all of the following *options* refer. If *printer* does not exist then it will be created.

The following *options* are only useful with `-p` and may appear in any order. For convenience, the printer will be referred to as *p*:

- `-cclass` Inserts the printer *p* into the specified *class*. *Class* will be created if it does not already exist.
- `-eprinter` Copies an existing *printer*'s interface program as the new interface program for *p*.
- `-h` Indicates that the device associated with *p* is hardwired. This *option* is assumed when creating a new printer unless the `-l` option is supplied.

- iinterface** Establishes a new interface program for *p*. *Interface* is the pathname of the new program.
- l** Indicates that the device associated with *p* is a login terminal. The lineprinter scheduler, *lpsched*(C), disables all login terminals automatically each time it is started. Before reenabling *p*, its current *device* should be established using *lpadmin*.
- rclass** Removes the printer *p* from the specified *class*. If *p* is the last member of the *class*, then the *class* will be removed.
- vdevice** Associates a new *device* with printer *p*. *Device* is the pathname of a file that is writable by the system manager, *lp*. Note that there is nothing to stop a system manager from associating the same *device* with more than one *printer*. If only the **-p** and **-v** options are supplied, then *lpadmin* may be used while the scheduler is running.

Restrictions

When creating a new printer, you must supply the **-v** option and one of the **-e**, **-i**, or **-m** options. Only one of the **-e**, **-i**, or **-m** options may be supplied. The **-h** and **-l** keyletters are mutually exclusive. Printer and class names may be no longer than 14 characters and must consist entirely of the A - Z, a - z, 0 - 9, and _ (underscore) characters.

Models

Model printer interface programs are shell procedures which interface between *lpsched*(C) and devices. Models should have 644 permission if owned by *lp* & bin, or 664 permission if owned by bin & bin. System managers may modify copies of models and then use *lpadmin -i* to associate them with printers. If printers have special options, these can be included in the interface program. You can then choose an option with the *lp -o* command.

Some model interface programs are supplied with lineprinter software, such as **dumb**, **Tandy.DMP**, and **emulator**.

Files

/usr/spool/lp/*

See Also

accept(C), enable(C), lp(C), lpinit(C), lpsched(C), 'pstat(C)

Name

`lpinit` – Adds new lineprinters to system.

Syntax

`/etc/lpinit`

Description

Lpinit is a shell script for configuring and adding new lineprinters to a system. It should only be executed by the system manager. *Lpinit* asks a series of questions for which the default answers are displayed. The system manager can type in a response or press the RETURN key for the default answer. *Lpinit* prompts users for the following information:

- The print device adapter (default is Serial/Parallel Adapter, Parallel Port)
- The character mode needed for the printer (default is non-interpretive*)
- The name of the printer (default is *linepr*).
- The pathname of the printer interface program (default is `/usr/spool/lp/model/dumb`).

* Non-interpretive mode does not process data sent to the printer. See “Special Handling for Tandy Printers” for more information.

The printer name can be any combination of up to 14 alphanumeric characters or underscores. A printer interface program can be a shell script, C program, or any executable program; or the model interface program, `/usr/spool/lp/model/dumb`, can be copied and modified.

After the system manager has responded to these questions, `/etc/lpinit` stops the print scheduler *lpsched*, changes the acceptance status of the new lineprinter to “accept,” and enables it to print files. `/etc/lpinit` then asks if the new printer will be the default printing destination (default is “Yes”). All nonspecific print requests are routed to the default destination (see *lp(C)*).

The steps to configure a new printer can be taken separately, (see *lpadmin(C)*, *accept(C)*, *enable(C)*, and *lpsched(C)* for more information) but you must use *lpinit* if the default printer is to be System III compatible.

Special Handling For Tandy Printers

Some printers need special handling for some codes. In interpretive mode the system sends LF as CR, TABs as the appropriate number of spaces, and FF (formfeeds) as the appropriate number of carriage returns.

In non-interpretive mode (default) the system sends every character to the printer unmodified.

The **lpinit** command asks you if you need “special handling for Tandy Carriage Returns and Line Feeds.” If you do not (non-interpretive mode), simply press RETURN in response to the prompt. If you do (interpretive mode), answer “y” to the prompt.

If your printer can be set (dip switches) to treat LF as NEWLINE and CR as CR (without a line feed) and the printer can do its own tabs and form feeds, you should use the non-interpretive mode. If you choose interpretive mode, the system will do translations of CR, FF, and TAB characters being sent to the printer. If you ask for these translations, please note the following:

- You must keep the printer’s actual top-of-form in sync with the driver’s idea of top-of-form.
- If you run a program that does any non-standard line spacing, such as half-line feeds or 8 lines per inch, the top-of-form will be off in subsequent output.
- If your output contains characters that are not uniformly spaced, the TAB translation may not work properly.

Files

/etc/lpinit

See Also

accept(C), enable(C), lp(C), lpadmin(C), lpsched(C)

Notes

If your printer cannot automatically do tabs, you can still use non-interpretive mode by using the **-e** option of the **pr** command when you print files that contain tabs.

Name

`lpsched`, `lpshut`, `lpmove` – Starts/stops the lineprinter request scheduler and moves requests.

Syntax

```
/usr/lib/lpsched  
/usr/lib/lpshut  
/usr/lib/lpmove requests destinations  
/usr/lib/lpmove dest1 dest2
```

Description

Lpsched schedules requests taken by *lp(C)* for printing on lineprinters.

Lpshut shuts down the lineprinter scheduler. All printers that are printing at the time *lpshut* is invoked will stop printing. Requests that were printing at the time a printer was shut down will be reprinted in their entirety after *lpsched* is started again. All lineprinter commands perform their functions even when *lpsched* is not running.

Lpmove moves requests that were queued by *lp(C)* between lineprinter destinations. This command may be used only when *lpsched* is not running. The first form of the command moves the named *requests* to the lineprinter *destinations*, *dest*. *Requests* are requested as returned by *lp(C)*. The second form moves all requests for the *dest1* destination to *dest2*. As a side effect, *lp(C)* will reject requests for *dest1*.

Note that *lpmove* never checks the acceptance status for the new destination when moving requests (see *accept(C)*).

Files

`/usr/spool/lp/*`

See Also

`accept(C)`, `enable(C)`, `lp(C)`, `lpadmin(C)`, `lpinit(C)`, `lpstat(C)`



Name

lpstat – Prints lineprinter status information.

Syntax

lpstat [*options...*]

Description

Lpstat prints information about the current status of the lineprinter system.

If no *options* are given, then *lpstat* prints the status of all requests made to *lp(C)* by the user. Any arguments that are not *options* are assumed to be request (as returned by *lp*). *Lpstat* prints the status of these requests. *Options* may appear in any order and may be repeated and intermixed with other arguments. Some of the following options may be followed by a *list* in one of two forms: a list of items separated from one another by commas, or a list of items enclosed in double quotation marks and separated from one another by a comma and one or more spaces. For example:

```
-u "user1, user2, user3"
```

The omission of a *list* following such options causes all information relevant to the option to be printed, for example:

```
lpstat -o
```

prints the status of all output requests.

-a[*list*] Prints acceptance status (with respect to *lp*) of destinations for requests. *List* is a list of intermixed printer names and class names.

-c[*list*] Prints class names and their members. *List* is a list of class names.

-d Prints the system default destination for *lp*.

-o[*list*] Prints the status of output requests. *List* is a list of intermixed printer names, class names, and request

-p[*list*] Prints the status of printers. *List* is a list of printer names.

-r Prints the status of the lineprinter scheduler, *lpsched*.

-s Prints a status summary, including the status of the lineprinter scheduler, the system default destination, a list of class names and their members, and a list of printers and their associated devices.

- t** Prints all status information.
- u[*list*]** Prints status of output requests for users. *List* is a list of login names.
- v[*list*]** Prints the names of printers and the pathnames of the devices associated with them. *List* is a list of printer names.

Files

/usr/spool/lp/*

See Also

enable(C), lp(C)

Name

`ls, ll, l, lc, lr, lx` – Gives information about contents of directories.

Syntax

`ls [ -1ACFRabcdgilmnopqrstux ] [ names ]`

Description

For each directory named, `ls` lists the contents of that directory; for each file named, `ls` repeats its name and any other information requested. By default, the output is sorted alphabetically. When no argument is given, the current directory is listed. When several arguments are given, the arguments are sorted appropriately. File arguments are processed before directories and their contents. The `ll` and `l` commands perform the same functions as `ls -l`.

There are three major listing formats. The default format is to list one entry per line, the `-C` and `-x` options enable multi-column formats, and the `-m` option enables stream output format in which files are listed across the page, separated by commas. In order to determine output format for the `-C`, `-x`, and `-m` options, `ls` uses an environment variable, `COLUMNS`, to determine the number of character positions available on one output line. If this variable is not set, the `termcap` database is used to determine the number of columns, based on the environment variable `TERM`. If this information cannot be obtained, 80 columns are assumed.

`Lc` lists the contents of files and directories, in columns. Output is given in columns and sorted alphabetically. Files that are not the contents of a directory being interpreted are always sorted across the page rather than down the page in columns. A stream output format is available in which files are listed across the page, separated by commas. The `-m` option enables this format. The `lr` and `lx` commands are equivalent to `lc -R` and `lc -x` respectively. The `l`, `ll`, `lc`, `lr` and `lx` commands are links to `ls`.

The `ls` and `lc` commands use the following options:

- `-1` Forces an output format with one entry per line.
- `-A` If the user is not the super-user, this option displays all files that begin with “.” (except “.” and “..” themselves). If the user is the super-user, this option suppresses display of all files that begin with “.”.
- `-C` Forces columnar output, even if redirected to a file.
- `-F` Causes directories to be marked with a trailing / and executable files to be marked with a trailing “*”.

- R Lists subdirectories recursively.
- a Lists all entries, including "." and "..", which are otherwise suppressed.
- b Forces printing of non-graphic characters to be in the octal \ddd notation.
- c Uses time of last modification of the inode (file created, mode, changed, etc.) for sorting (-t) or printing (-l).
- d If an argument is a directory, lists only its name (not its contents); often used with -l to get the status of a directory.
- f Forces each argument to be interpreted as a directory and lists the name found in each slot. This option turns off -l, -t, -s, and -r, and turns on -a; the listing order is the order in which entries appear in the directory.
- g The same as -l, except that the owner is not printed.
- i Prints the inode number in first column of the report for each file listed.
- l Lists in long format, giving mode, number of links, owner, group, size in bytes, and time of last modification for each file. If the file is a special file, the size field contains the major and minor device numbers instead.
- m Forces stream output format.
- n Same as the -l switch, but the owner and group ID 's appear instead of the owner and group names. This option has the same effect when used with the -g and -o options.
- o The same as -l, except that the group is not printed.
- p Puts a slash (/) after each filename if that file is a directory.
- q Forces printing of nongraphic characters in filenames as the ? character.
- r Reverses the order of sorting to get either reverse alphabetic or oldest first sorting, as appropriate.
- s Gives size in 512 byte blocks, including indirect blocks for each entry.
- t Sorts by time modified (latest first) instead of by name, as is normal.
- u Uses time of last access instead of last modification for sorting (-t) or printing (-l).
- x Forces columnar printing to be sorted across rather than down the page.

The mode printed under the `-l` option consists of 11 characters. The first character is:

- If the entry is an ordinary file.
- d** If the entry is a directory.
- b** If the entry is a block special file.
- c** If the entry is a character special file.
- p** If the entry is a named pipe.
- s** If the entry is a XENIX semaphore.
- m** If the entry is a XENIX shared data (memory) file.

The next nine characters are interpreted as three sets of three bits each. The first set refers to the owner's permissions; the next to permissions of others in the user-group of the file; and the last to all others. Within each set, the three characters indicate permission to read, to write, and to execute the file as a program, respectively. For a directory, "execute" permission is interpreted to mean permission to search the directory for a specified file.

The permissions are indicated as follows:

- r** If the file is readable.
- w** If the file is writeable.
- x** If the file is executable.
- If the indicated permission is *not* granted.

The group-execute permission character is given as "s" if the file has set-group-ID mode; likewise, the user-execute permission character is given as "S" if the file has set-user-ID mode.

The last character of the mode (normally **x** or **-**) is **t** if the 1000 (octal) bit of the mode is on; see `chmod(C)` for the meaning of this mode. The indications of set-ID and 1000 bit of the mode are capitalized if the corresponding execute permission is *not* set.

When the sizes of the files in a directory are listed, a total count of blocks including indirect blocks is printed.

Files

/etc/passwd	Gets user IDs for <i>ls -l</i> and <i>ls -o</i>
/etc/group	Gets group IDs for <i>ls -l</i> and <i>ls -g</i>
/etc/termcap	Gets terminal information

See Also

chmod(C), find(C), termcap(C)

Notes

NEWLINE and TAB are considered printing characters in filenames.

Unprintable characters in filenames may confuse the columnar output options.

Column width choices are poor for terminals that can TAB.

Credit

This utility was developed at the University of California at Berkeley and is used with permission.

Name

mail – Sends, reads or disposes of mail.

Syntax

```
mail [[ -u user ] [ -f mailbox ]] [ -e ] [ -R ] [ -i ] [ users ... ]
```

```
mail [ -s subject ] [ -i ] [ user ... ]
```

Description

Mail is a mail processing system that supports composing of messages, and sending and receiving of mail between multiple users. When sending mail, a *user* is the name of a user or of an alias assigned to a machine or to a group of users.

Options include:

- u *user*** Tells *mail* to read the system mailbox belonging to the specified *user*.
- f *mailbox*** Tells *mail* to read the specified *mailbox* instead of the default user's system mailbox.
- e** Allows escapes from compose mode when input comes from a file.
- R** Makes the mail session “read-only” by preventing alteration of the mailbox being read. Useful when accessing system-wide mailboxes.
- i** Tells *mail* to ignore interrupts sent from the terminal. This is useful when reading or sending mail over telephone lines where “noise” may produce unwanted interrupts.
- s *subject*** Specifies *subject* as the text of the *Subject:* field for the message being sent.

Sending mail

To send a message to one or more other people, invoke *mail* with arguments which are the names of people to send to. You are then expected to type in your message, followed by a CTRL-D at the beginning of a line.

Reading Mail

To read mail, invoke *mail* with no arguments. This will check your mail out of the system-wide directory so that you can read and dispose of the messages sent to you. A message header is printed out for each message in your mailbox. The current message is initially the last numbered message and can be printed using the *print* command (which can be abbreviated *p*). You can move among the messages

much as you move between lines in *ed*, with the + and - commands moving backwards and forwards, and simple numbers typing the addressed message.

If new mail arrives during the mail session you can read in the new messages with the *restart* command.

Disposing of Mail

After examining a message you can *delete* (*d*) the message or *reply* (*r*) to it. Deletion causes the *mail* program to forget about the message. This is not irreversible; the message can be *undeleted* (*u*) by giving its number, or the *mail* session can be aborted by giving the *exit* (*x*) command. Usually, though, deleted messages will disappear completely.

Specifying Messages

Commands such as *print* and *delete* often can be given a list of message numbers as arguments to apply to a number of messages at once. Thus, “delete 1 2” deletes messages 1 and 2, while “delete 1-5” deletes messages 1 through 5. The special name “*” addresses all messages, and “\$” addresses the last message; thus, the *top* command, which prints the first few lines of a message, could be used in “top *” to print the first few lines of all messages.

Replying to or Originating Mail

You can use the *reply* command to set up a response to a message, sending it back to the person who sent it. Then you can type in the text of the reply, and press CTRL-D to send it. While you are composing a message, *mail* treats lines beginning with a tilde (~) as special. For instance, typing “~m” (alone on a line) places a copy of the current message into the response, shifting it right by one tabstop. Other escapes set up subject fields, add and delete recipients to the message, and allow you to escape either to an editor to revise the message or to a shell to run some commands. (These options are summarized later.)

Ending a Mail Session

You can end a *mail* session with the *quit* (*q*) command. Messages that have been examined go to your *mbox* file unless they have been deleted. Unexamined messages go back to the post office. The -f option causes *mail* to read in the contents of your *mbox* (or the specified file) for processing; when you *quit*, *mail* writes undeleted messages back to this file. The -i option causes *mail* to ignore interrupts.

Using Aliases and Distribution Lists

It is also possible to create a personal distribution list. For instance, you can send mail to “cohorts” and have it go to a group of people. Such lists can be defined by typing a line like:

```
alias cohorts bill bob barry bobo betty beth bobbi
```

in the `.mailrc` file of your home directory. The current list of such aliases can be displayed by the *alias* (*a*) command in *mail*. System-wide distribution lists can be created by editing `/usr/lib/mail/aliases`; see *aliases*(M); these are kept in a slightly different syntax. In mail that you send, personal aliases will be expanded in mail sent to others so that they will be able to *reply* to the recipients. System-wide *aliases* are not expanded when the mail is sent, but any reply returned to the machine *will* have the system-wide alias expanded.

Mail has a number of options which can be *set* in the `.mailrc` file to alter its behavior; thus, “set askcc” enables the “askcc” feature. (These options are summarized below.)

Summary

Each mail command is typed on a line by itself, and may take arguments following the command word. The command need not be typed in its entirety – the first command which matches the typed prefix is used. For the commands that take message lists as arguments, if no message list is given, then the next message forward that satisfies the command’s requirements is used. If there are no messages forward of the current message, the search proceeds backwards, and if there are no messages at all, *mail* displays “No applicable messages” and aborts the command.

- Goes to the previous message and prints it out. If given a numeric argument, *n*, goes to the *n*th previous message and prints it.
- +
- Goes to the next message and prints it out. If given a numeric argument, *n*, goes to the *n*th next message and prints it.
- RETURN Goes to the next message and prints it out.
- ? Prints a brief summary of commands.
- ! Executes the shell command which follows.
- = Prints out the current message number.
- ^ Prints out the first message.
- \$ Prints out the last message.

alias	(a) With no arguments, prints out all currently-defined aliases. With one argument, prints out that alias. With more than one argument, adds the users named in the second and later arguments to the alias named in the first argument.
Alias users	Prints system-wide list of aliases for users. At least one user must be specified.
cd	(c) Changes the user's working directory to that specified. If no directory is given, it changes to the user's login directory.
delete	(d) Takes a list of messages as an argument and marks them all as deleted. Deleted messages are not retained in the system mailbox after a quit, nor are they available to any command other than the <i>undelete</i> command.
dp	Deletes the current message and prints the next message. If there is no next message, <i>mail</i> says "No more messages."
echo PATH	Expands shell metacharacters.
edit	(e) Takes a list of messages and points the text editor at each one in turn. Upon returning from the editor, the message is read back in.
exit	(x) Effects an immediate return to the shell without modifying the user's system mailbox, <i>mbox</i> file, or <i>edit</i> file in <i>-f</i> .
file	(fi) Prints the name of the file <i>mail</i> is reading. If it is a mailbox, the name of the owner is returned.
forward	(f) Forwards the current message to the named users. The current message is indented within the forwarded message.
Forward	(F) Forwards the current message to the named users. The current message is <i>not</i> indented within the forwarded message.
headers	(h) Lists the current range of headers, which is an 18 message group. If a "+" argument is given, then the next 18 message group is printed, and if a "-" argument is given, the previous 18 message group is printed. Both "+" and "-" may take a number to view a particular window. If a message-list is given, it prints the specified headers.
hold	(ho) Takes a message list and marks each message to be saved in the user's system mailbox instead of in <i>mbox</i> . Use only when the <i>autombox</i> switch is set. Does not override the <i>delete</i> command.
list	Prints a list of <i>mail</i> commands.

- lpr** (l) Prints out each message in a message-list on the lineprinter.
- mail** (m) Takes as argument login names and distribution group names and sends mail to those people.
- mbox** (mb) Marks messages in a message list so that they are saved in the user mailbox after leaving mail.
- move** *mesg-list mesg-num*
Places the messages specified in *mesg-list* after the message specified in *mesg-num*. If *mesg-num* is 0, *mesg-list* moves to the top of the mailbox.
- next** (n like + or RETURN) Goes to the next message in sequence and prints it. With an argument list, *next* types the next matching message.
- print** (p) Prints out each message in a message-list on the terminal display.
- quit** (q) Terminates the session, retaining all undeleted, unsaved messages in the system mailbox and removing all other messages. Files marked with a star (*) are saved; files marked with an "M" are saved in the user mailbox. If new mail has arrived during the session, the message "You have new mail" is given. If quit is given while editing a mailbox file with the -f flag, then the edit file is rewritten. You then return to the shell, unless the rewrite of the edit file fails, in which case you can escape with the *exit* command.
- reply** (r) Takes a message list and sends mail to each message author. The default message must not be deleted.
- Reply** (R) Takes a message list and sends mail to each message author *and each member of the message* just like the *mail* command. The default message must not be deleted.
- restart** Reads in messages that arrived during the current mail session.
- save** (s) Takes a message list and a filename and appends each message in turn to the end of the file. The filename, in quotation marks, followed by the line count and character count is echoed on the user's terminal.
- set** (se) With no arguments, prints all variable values. Otherwise, sets option. Arguments are of the form "option=value" or "option".
- shell** (sh) Invokes an interactive version of the shell.

size	(<i>si</i>) Takes a message list and prints out the size in characters of each message.
source	(<i>so</i>) Reads mail commands from the file given as its only argument.
string <i>string mesg-list</i>	Searches for <i>string</i> in <i>mesg-list</i> . If no <i>mesg-list</i> is specified, all undeleted messages are searched. Case is ignored in search.
top	(<i>t</i>) Takes a message list and prints the top few lines of each. The number of lines printed is controlled by the <i>toplines</i> variable and defaults to six.
undelete	(<i>u</i>) Takes a message list and marks each one as <i>not</i> being deleted.
unset	(<i>uns</i>) Takes a list of option names and discards their remembered values; the inverse of <i>set</i> .
visual	(<i>v</i>) Takes a message list and invokes <i>vi</i> on each message.
write <i>filename</i>	(<i>w</i>) Saves the body of the message in the named file.

Here is a summary of the compose escapes, which are used when composing messages to perform special functions. Compose escapes are only recognized at the beginning of lines.

~ <i>string</i>	Inserts the string of text in the message prefaced by a single tilde (~). If you have changed the ESCAPE character, then you should double that character instead.
~ ?	Prints out help for compose escapes.
~ .	Same as CTRL-D on a new line.
~ ! <i>cmd</i>	Executes the indicated shell command, then returns to the message.
~ <i>cmd</i>	Pipes the message through the command as a filter. If the command gives no output or terminates abnormally, retains the original text of the message.
~ _ <i>mail-command</i>	Executes a mail command, then returns to compose mode.
~ : <i>mail-command</i>	Executes a mail command, then returns to compose mode.
~ alias	Prints list of private aliases

- `~ alias aliasname` Prints names included in private *aliasname*.
- `~ Alias` Performs aliasing by first examining private aliases and then system-wide aliases using all three global alias files (*aliases.hash*, *faliases*, and *maliases*). Only the final result is printed (non-local mail recipients will have the complete delivery path printed). The user list is taken from header fields.
- `~ Alias users` Performs aliasing by first examining private aliases and then system-wide aliases using all three global alias files (*aliases.hash*, *faliases*, and *maliases*). Only the final result is printed (non-local mail recipients will have the complete delivery path printed). At least one user must be specified.
- `~ b name ...` Adds the given names to the list of blind carbon copy recipients.
- `~ c name ...` Adds the given names to the list of carbon copy recipients.
- `~ cc name ...` Same as `~ c`.
- `~ d` Reads the *dead.letter* file from your home directory into the message.
- `~ e` Invokes the text editor on the message collected so far. After the editing session is finished, you may continue appending text to the message.
- `~ h` Edits the message header fields by typing each one in turn and allowing the user to append text to the end or modify the field with the current terminal ERASE and KILL characters.
- `~ m mesg-list` Reads the named messages into the message buffer, shifted right one TAB. If no messages are specified, reads the current message.
- `~ M mesg-list` Reads the named messages into the message buffer, shifted right one TAB. If no messages are specified, reads the current message.
- `~ p` Prints out the messages collected so far, prefaced by the message header fields.
- `~ Print` Prints the real names or descriptions (in parentheses) after each recipient in a header field.
- `~ q` Aborts the message being sent, copying the message to the *dead.letter* file in your home directory if *save* is set.
- `~ r filename` Reads the named file into the message buffer. Do not use this escape sequence in compose mode.

- `- Return name` Adds the given names to the Return-receipt-to field.
- `- s string` Causes the named string to become the current subject field.
- `- t name ...` Adds the given names to the direct recipient list.
- `- v` Invokes a visual editor (defined by the **VISUAL** option) on the message buffer. After you quit the editor, you may resume appending text to the end of your message.
- `- w filename` Writes the body of the message to the named file.

Options are controlled with the *set* and *unset* commands. An option may be either a switch, in which case it is either on or off, or a string, in which case the actual value is of interest. The switch options include the following:

- askcc** Causes you to be prompted for additional carbon copy recipients at the end of each message. Responding with a NEWLINE indicates your satisfaction with the current list.
- asksubject** Causes *mail* to prompt you for the subject of each message you send. If you respond with simply a NEWLINE, no subject field is sent.
- autombox** Causes all examined messages to be saved in the user mailbox unless deleted or saved.
- autoprint** Causes the *delete* command to behave like *dp* – thus, after deleting a message, the next one will be typed automatically.
- chron** Causes messages to be displayed in chronological order. The **chron** option will not work when **mchron** is set.
- dot** Permits use of dot (.) as the end-of-file character when composing messages.
- ignore** Causes interrupt signals from your terminal to be ignored and echoed as at-signs (@).
- mchron** Causes messages to be listed in numerical order (most recently received first), but displayed in chronological order. The **mchron** option does not work will **chron** is set.
- metoo** Usually, when a group is expanded that contains the sender, the sender is removed from the expansion. Setting this option causes the sender to be included in the group.

nosave	Prevents aborted messages from being appended to the dead.letter file in your home directory on receipt of two interrupts (or a <code>~q</code>).
quiet	Suppresses the printing of the version header when first invoked.
verify	Causes each target mail recipient to be verified. This option permits errors made while composing messages to be corrected or ignored.

The following options have string values:

EDITOR	Pathname of the text editor to use in the <i>edit</i> command and <code>~e</code> escape. If not defined, then a default editor is used.
SHELL	Pathname of the shell to use in the <i>!</i> command and the <code>~!</code> escape. A default shell is used if this option is not defined.
VISUAL	Pathname of the text editor to use in the <i>visual</i> command and <code>~v</code> escape.
escape	If defined, the first character of this option gives the character to use in the place of the tilde (<code>~</code>) to denote escapes.
page=<i>n</i>	Specifies the number of lines (<i>n</i>) to be printed in a “page” of text when displaying messages.
record	If defined, gives the pathname of the file used to record all outgoing mail. If not defined, then outgoing mail is not saved.
toplines	If defined, gives the number of lines of a message to be printed out with the <i>top</i> command; normally, the first six lines are printed.

Files

<code>/usr/spool/mail/*</code>	System mailboxes
<code>/usr/name/dead.letter</code>	File where undeliverable mail is deposited
<code>/usr/name/mbox</code>	Your old mail
<code>/usr/name/.mailrc</code>	File giving initial mail commands
<code>/usr/lib/mail/aliases</code>	System-wide aliases

/usr/lib/mail/aliases.hash	System-wide alias database
/usr/lib/mail/faliases	Forwarding aliases for the local machine
/usr/lib/mail/maliases	Machine aliases
/usr/lib/mailhelp.cmd	Help file
/usr/lib/mailhelp.esc	Help file
/usr/lib/mailhelp.set	Help file
/usr/lib/mail/mailrc	System initialization file
/usr/bin/mail	The <i>mail</i> command

Notes

The default mailbox created by the `mkuser` command has permission settings of 644. It is recommended that you change the permission setting to avoid an obvious security hole.

At command mode, if you save, delete, and undelete a message, the **M** flag (save in mailbox) is set.

This utility was developed at the University of California at Berkeley and is used with permission.

Bugs

Aliases are not supported in lists.

See Also

`aliases(M)`, `aliashash(M)`, `mkuser(C)`, `netutil(C)`

Name

makefs - Makes a file system.

Description

The *makefs* command is a menu driven script that lets you easily create file systems on a floppy disk, fixed disk, or disk cartridge. To invoke *makefs*, type:

```
makefs
```

and press the RETURN key. Your screen shows the following menu:

```
Make a File System on...
-----
1      a floppy disk
2      a hard disk
3      a disk cartridge

q      quit
```

Floppy Disk

To create a file system on a floppy disk, type 1 and press the RETURN key. The *makefs* command asks you to enter the drive number on which you want to make the file system. Enter the appropriate number and press the RETURN key.

You are then prompted to specify the format of the floppy diskette. Choose one of the displayed formats and press the RETURN key.

Your screen shows:

```
Do you wish to format this disk (y/n)?
```

If your floppy disk is not formatted, type 'Y' and press the RETURN key. The *makefs* command prompts you to insert the floppy disk and then formats it. After formatting is complete, *makefs* creates a file system on the floppy disk.

If the disk is already formatted, type 'N' and press the RETURN key. The *makefs* command creates a file system on the floppy disk.

Fixed Disk

Before creating a file system on a secondary fixed disk, be sure the disk is formatted, contains a bad track table, and is partitioned.

To create a file system on a secondary hard disk, type '2' and press the RETURN key. The *makefs* command asks you to enter the drive and partition number on which you want to make the file system. Enter the appropriate numbers and press the RETURN key.

The *makefs* command creates a file system on the fixed disk.

Disk Cartridge

To create a file system on a disk cartridge, type 3 and press the RETURN key. *Makefs* asks you to enter the drive number on which you want to make the file system. Enter the appropriate number and press the RETURN key.

Your screen shows:

Do you wish to format this disk (y/n)?

If your disk cartridge is not formatted, type 'Y' and press the RETURN key. *Makefs* prompts you to insert the disk cartridge and then formats it. After formatting is complete, *makefs* creates a file system on the disk cartridge.

If the disk is already formatted, type 'N' and press the RETURN key. *Makefs* creates a file system on the disk cartridge.

Notes

Makefs is a super-user only command.

You cannot create a file system on a tape cartridge.

Error messages only occur when using *makefs* on block devices.

XENIX locks disk cartridges while they are in use. This is to prevent anyone from removing a disk while it is in motion. During this time, the cartridge release button is ignored. When the drive is no longer in use, XENIX unlocks the drive so you can remove the disk cartridge.

If you are using the disk cartridge as a mountable file system, XENIX keeps the drive locked until you unmount the file system. Note that if you improperly shut-down your XENIX system or if it crashes with the cartridge mounted or in use, XENIX does not unlock the drive. To unlock it, press the reset switch. XENIX unlocks the disk cartridge at the XENIX Boot prompt so you can remove the cartridge.

Name

`msg` – Permits or denies messages sent to a terminal.

Syntax

`msg [ n ] [ y ]`

Description

Msg with the `n` argument forbids messages via *write*(C) by revoking nonuser write permission on the user's terminal. *Msg* with the `y` argument reinstates this permission. All by itself, *msg* reports the current state without changing it.

Files

`/dev/tty*`

See Also

write(C)

Diagnostics

The exit status is 0 if messages are receivable, 1 if they are not, 2 if there is an error.



Name

mkdir – Makes a directory.

Syntax

mkdir *dirname* ...

Description

Mkdir creates directories. The standard entries “dot” (.), for the directory itself, and “dot dot” (..), for its parent directory, are made automatically.

Mkdir requires write permission in the parent directory. The permissions assigned to the new directory are modified by the current file creation mask set by *umask*(C).

See Also

`rmdir(C)`, `umask(C)`

Diagnostics

Mkdir returns an exit code of 0 if all directories were successfully made; otherwise, it prints a diagnostic and returns nonzero.



Name

mkfs – Constructs a file system.

Syntax

```
/etc/mkfs [-y] [-n] special blocks [: inodes] [gap blocks]  
/etc/mkfs [-y] [-n] special proto [gap blocks]
```

Description

Mkfs constructs a file system by writing on the special file according to the directions found in the remainder of the command line.

If it appears that the special file contains a file system, operator confirmation is requested before overwriting the data. The **-y** (yes) option overrides this, and writes over any existing data without question. The **-n** (no) option causes *mkfs* to terminate without question if the target contains an existing file system. The check used is to read block one from the target device (block one is the super-block) and see whether the bytes are the same. If they are not, this is taken to be meaningful data and confirmation is requested. The number of bytes in a block is BSIZE as defined in `/usr/include/sys/param.h`.

If the second argument is given as a string of digits, *mkfs* builds a file system with a single empty directory on it. The size of the file system is the value of *blocks* interpreted as a decimal number. The boot program is left uninitialized. If the number of inodes is specified, then this number should be the same as the estimated number of files in the file system. If the optional number of inodes is not given, the number of inodes is calculated as a function of the system file size.

If the second argument is a file name that can be opened, *mkfs* assumes it to be a prototype file **proto**, and will take its directions from that file. The prototype file contains tokens separated by spaces or NEWLINES. The first token is the name of a file to be copied onto block zero as the bootstrap program. The bootstrap program specified should already be stripped of the header (see *strip*(CP)). If the header has not been stripped from the bootstrap program, then *mkfs* will issue a warning. The second token is a number specifying the size of the created file system. Typically it will have been the number of blocks on the device, perhaps diminished by space for swapping. The next token is the *i*-list size in blocks. The next set of tokens comprise the specification for the root file. File specifications consist of tokens giving the mode, the user ID, the group ID, and the initial contents of the file. The syntax of the contents field depends on the mode.

The mode token for a file is a six character string. The first character specifies the type of the file. (The **-bcd** characters specify regular, block special, character special, and directory files, respectively.) The second character of the type is either **u** or **-** to specify set-user-ID mode or not. The third character is either **g** or **-** for the set-group-ID mode. The rest of the mode is a three digit octal number giving the owner, group, and other read, write, and execute permissions (see *chmod*(C)).

Two decimal number tokens come after the mode; they specify the user and group ID's of the owner of the file.

If the file is a regular file, the next token is a pathname where the contents and size of the file are copied. If the file is a block or character special file, two decimal number tokens follow which give the major and minor device numbers. If the file is a directory, *mkfs* makes the entries `.` and `..` and then reads (recursively) a list of names and file specifications for the entries in the directory. The scan is terminated with the `$` token.

A sample prototype specification follows:

```
/stand/diskboot
4872 110
d--777 3 1
usr d--777 3 1
  sh ---755 3 1 /bin/sh
  ken d--755 6 1
    $
  b0 b--644 3 1 0 0
  c0 c--644 3 1 0 0
    $
  $
```

In both command syntaxes, the disk interleaving factors, *gap* and *blocks*, can be specified. The interleaving factors are a function of the disk hardware and are described in detail in the *XENIX Operations Guide*.

See Also

`chmod(C)`, `dir(F)`, `filesystem(F)`, `strip(CP)`

Notes

There is no way to specify links when using a prototype file. If the number of *inodes* is specified on the command line, then the maximum number of *inodes* in the file system is 65500.

Name

`tail` – Delivers the last part of a file.

Syntax

`tail [ ±[ number ][ lbc ] [ -f ] ] [ file ]`

Description

Tail copies the named *file* to the standard output beginning at a designated place. If no file is named, the standard input is used.

Copying begins at distance *+number* from the beginning, or *-number* from the end of the input (if *number* is null, the value 10 is assumed). *Number* is counted in units of lines, blocks (512 bytes each), or characters, according to the appended option, **l**, **b**, or **c**. When no units are specified, counting is by lines.

With the `-f` (follow) option, if the input file is not a pipe, the program will not terminate after the line of the input file has been copied, but it will enter an endless loop. In this loop, the program sleeps for a second and then attempts to read and copy further records from the input file. Thus, it may be used to monitor the growth of a file that is being written by some other process. For example, the command:

```
tail -f file
```

will print the last ten lines of *file*, followed by any lines that are appended to *file* between the time *tail* is initiated and killed.

See Also

`dd`(C)

Notes

Tails relative to the end of the file are kept in a buffer, and thus are limited in length. Unpredictable results can occur if character special files are “tailed”.



Name

mknod – Builds special files.

Syntax

/etc/mknod name [c] [b] major minor

/etc/mknod name p

/etc/mknod name s

/etc/mknod name m

Description

Mknod makes a directory entry and corresponding inode for a special file. The first argument is the *name* of the entry. In the first case, the second argument is **b** if the special file is block-type (disks, tape) or **c** if it is character-type (other devices). The last two arguments are numbers specifying the *major* device type and the *minor* device (e.g., unit, drive, or line number), which may be either decimal or octal.

The assignment of major device numbers is specific to each system. They can be found in **conf.c**, the system source file.

Mknod can also be used to create named pipes with the **p** option; XENIX semaphores with the **s** option; and XENIX shared data (memory) with the **m** option.

Only the super-user can use the first form of the syntax.

System Compatibility

The **s** and **m** options can only be used to create XENIX semaphores and shared data, not UNIX System V semaphores and shared data.

See Also

mknod(S)



Name

mkuser – Adds a login ID to the system.

Syntax

/etc/mkuser

Description

Mkuser is used to add more user login IDs to the system. It is the preferred method for adding new users to the system, since it handles all directory creation and password file update. To add a new user to the system, *mkuser* requires four pieces of information: the login name, the initial password, the group identification, and an optional comment string for the password file. It also allows the new user to be assigned to a group if required, although in most cases a default group is suitable. The program prompts for these four items and validates the given data. The login name is checked against certain criteria (i.e., it must be at least three characters and begin with a lowercase letter). The password must follow standard XENIX conventions, see *passwd(8)*. The password file comment field can be up to 255 characters of information.

Mkuser takes some of its parameters from a default file, */etc/default/mkuser*. Currently, one may set the root path of home directories. An example default file is:

```
HOME=/usr
```

This file can be edited (by the super-user) to change this default. There are three other files in the directory */usr/lib* which may also be altered to suit local options. They are *mkuser.help* which is the introductory explanation given by *mkuser* on startup, *mkuser.mail* which is the initial mail message sent to new users, and *mkuser.profile*, the standard *.profile* file given to new users.

Mkuser allocates user IDs starting at 200, or the largest number used in the password file. The default group ID for new users is 50. The minimum group ID allowed for user accounts is 50. The program prompts the operator for an optional group specification. This can either be a numeric group ID, or a group name. If the group exists the user is added to it. If it does not exist, a new entry in */etc/group* is created. A new group cannot have a numeric ID less than 51. If a new group is to be created, and the operator only specifies the group name, a free group ID is assigned. Alternatively, the operator can specify the group ID too.

Mkuser can only be executed by the super-user.

The minimum length of a legal password, and the minimum and maximum number of weeks used in password aging are specified in */etc/default/passwd* by the variables *PASSLENGTH*, *MINWEEKS*, and *MAXWEEKS*. For example, these variables might be set as follows:

```
PASSLENGTH=6  
MINWEEKS=2  
MAXWEEKS=6
```

Files

```
/etc/passwd  
/usr/spool/mail/username  
/etc/default/mkuser  
/usr/lib/mkuser.cshrc  
/usr/lib/mkuser.help  
/usr/lib/mkuser.login  
/usr/lib/mkuser.prof  
/usr/lib/mkuser.mail
```

Notes

Passwords created by the *mkuser* command, as represented in the */etc/passwd* file, are not guaranteed to be unique. This is because the encrypting algorithm does not use the PID and time of day clock to determine the encrypting keyword. Therefore, it is recommended that the new user change his password immediately after login in, using the *passwd(C)* command.

See Also

default(M), *passwd(C)*, *pwadmin(C)*, *rmuser(C)*

Name

more – Views a file one full screen at a time.

Syntax

more [**-cdfslur**] [**-n**] [**+linenumber**] [**+/pattern**] [**name ...**]

Description

This filter allows examination of a continuous text one full screen at a time. It normally pauses after each full screen, printing “--More--” at the bottom of the screen. If the user then types RETURN, one more line is displayed. If the user hits the SPACEBAR, another full screen is displayed. Other possibilities are described below.

The command line options are:

- n** An integer which is the size (in lines) of the window which *more* will use instead of the default.
- c** *More* draws each page by beginning at the top of the screen and erasing each line just before it draws on it. This avoids scrolling the screen, making it easier to read while *more* is writing. This option is ignored if the terminal does not have the ability to clear to the end of a line.
- d** *More* prompts with the message “Hit space to continue, Del to abort” at the end of each full screen. This is useful if *more* is being used as a filter in some setting, such as a class, where many users may be inexperienced.
- f** This option causes *more* to count logical lines, rather than screen lines. That is, long lines are not folded. This option is recommended if *nroff* output is being piped through *ul*, since the latter may generate escape sequences. These escape sequences contain characters that would ordinarily occupy screen positions, but that do not print when they are sent to the terminal as part of an escape sequence. Thus, *more* may think that lines are longer than they actually are and fold lines erroneously.
- l** Does not treat CTRL-L (FORMFEED) specially. If this option is not given, *more* pauses after any line that contains a CTRL-L, as if the end of a full screen had been reached. Also, if a file begins with a FORMFEED, the screen is cleared before the file is printed.

- s** Squeezes multiple blank lines from the output, producing only one blank line. Especially helpful when viewing *nroff* output, this option maximizes the useful information present on the screen.
- u** Normally, *more* handles underlining, such as that produced by *nroff* in a manner appropriate to the particular terminal: if the terminal can perform underlining or has a stand-out mode, *more* outputs appropriate escape sequences to enable underlining or stand-out mode for underlined information in the source file. The **-u** option suppresses this processing.
- r** Normally, *more* ignores control characters that it does not interpret in some way. The **-r** option causes these to be displayed as `^C`, where “C” stands for any such control character.
- w** Normally, *more* exits when it comes to the end of its input. With **-w**, however, *more* prompts and waits for any key to be struck before exiting.
- +linenumber** Starts up at *linenumber*.
- +/pattern** Starts up two lines before the line containing the regular expression *pattern*.

More looks in the `/etc/termcap` file for the terminal characteristics and the default window size. For example, on a terminal capable of displaying 24 lines, the default window size is 22 lines.

More looks in the environment variable, `MORE`, to preset any flags desired. For example, if you prefer to view files using the **-c** mode of operation, the shell command, `“MORE=-c”`, in the `.profile` file causes all invocations of *more* to use this mode.

If *more* is reading from a file, rather than a pipe, then a percentage is displayed along with the `“--More--”` prompt. This gives the fraction of the file (in characters, not lines) that has been read so far.

Other sequences which may be typed when *more* pauses, and their effects, are as follows (*i* is an optional integer argument, defaulting to 1):

- i*<space>** Displays *i* more lines, (or another full screen if no argument is given).
- CTRL-D** Displays 11 more lines (a “scroll”). If *i* is given, then the scroll size is set to *i*.
- d** Same as CTRL-D.

<i>iz</i>	Same as typing a space except that <i>i</i> , if present, becomes the new window size.
<i>is</i>	Skips <i>i</i> lines and prints a full screen of lines.
<i>if</i>	Skips <i>i</i> full screens and prints a full screen of lines.
q or Q	Exits from <i>more</i> .
=	Displays the current line number.
v	Starts up the screen editor <i>vi</i> at the current line. (Note that <i>vi</i> may not be available with your system.)
h or ?	Help command; Gives a description of all the <i>more</i> commands.
<i>!expr</i>	Searches for the <i>i</i> th occurrence of the regular expression <i>expr</i> . If there are less than <i>i</i> occurrences of <i>expr</i> , and the input is a file (rather than a pipe), then the position in the file remains unchanged. Otherwise, a full screen is displayed, starting two lines before the place where the expression was found. The user's ERASE and KILL characters may be used to edit the regular expression. Erasing back past the first column cancels the search command.
<i>in</i>	Searches for the <i>i</i> th occurrence of the last regular expression entered.
'	(Single quotation mark) Goes to the point from which the last search started. If no search has been performed in the current file, this command goes back to the beginning of the file.
<i>!command</i>	Invokes a shell with <i>command</i> . The % and ! characters in "command" are replaced with the current filename and the previous shell command respectively. If there is no current filename, % is not expanded. The sequences, "% " and "! ", are replaced by "% " and "! ", respectively.
<i>i:n</i>	Skips to the <i>i</i> th next file given in the command line (skips to last file if <i>n</i> doesn't make sense).
<i>i:p</i>	Skips to the <i>i</i> th previous file given in the command line. If this command is given in the middle of printing out a file, <i>more</i> goes back to the beginning of the file. If <i>i</i> doesn't make sense, <i>more</i> skips back to the first file. If <i>more</i> is not reading from a file, the bell rings and nothing else happens.
:f	Displays the current filename and line number.

:q or :Q Exits from *more* (same as *q* or *Q*).

. Repeats the previous command.

The commands take effect immediately, i.e., it is not necessary to type a RETURN. Up to the time when the command character itself is given, you may press the KILL character to cancel the numerical argument being formed. In addition, you may press the ERASE character to redisplay the "--More--(xx%)" message.

The terminal is set to *noecho* mode by this program so that the output can be continuous. Then, what you type will not show on your terminal, except for the slash (/) and exclamation (!) commands.

If the standard output is not a teletype, *more* acts just like *cat*, except that a header is printed before each file (if there is more than one file).

A sample usage of *more* in previewing *nroff* output would be:

```
nroff -ms +2 doc.n | more -s
```

Files

/etc/termcap Terminal database

/usr/lib/more.help Help file

See Also

csh(CP), environ(M), sh(C)

Notes

The *vi* and *help* options may not be available.

Before displaying a file, *more* attempts to detect whether it is an unprintable binary file such as a directory or an executable binary image. If *more* concludes that a file is unprintable, it refuses to print it. However, *more* cannot detect all possible kinds of unprintable files.

This utility was developed at the University of California at Berkeley and is used with permission.

Bugs

The *more* command is not able to figure out that standard output has been redirected. Normally, *more* tries to figure out if its output is a tty. If it is, *more* prints the "--more--" message and does magic things with the terminal modes. If it is not, *more* works silently.

Name

`mount` – Mounts a file structure.

Syntax

`/etc/mount [ special-device directory [ -r ]]`

`/etc/umount special-device`

Description

Mount announces to the system that a removable file structure is present on *special-device*. The file structure is mounted on *directory*. The *directory* must already exist; it becomes the name of the root directory of the newly mounted file structure.

The *mount* and *umount* commands maintain a table of mounted devices. If it is invoked with no arguments, for each *special-device*, *mount* prints the name of the device, the directory name of the mounted file structure, whether the file structure is read-only, and the date it was mounted.

The optional last argument indicates that the file is to be mounted as read-only. Physically write-protected files must be mounted in this way or errors will occur when access times are updated, whether or not any explicit write is attempted.

Umount removes the file structure previously mounted on device *special-device*.

Files

`/etc/mnttab` Mount table

See Also

`mnttab(F)`, `mount(S)`, `umount(C)`

Diagnostics

Mount issues a warning if the file structure to be mounted is currently mounted under another name.

Busy file structures cannot be dismounted with *umount*. A file structure is busy if it contains an open file or some user's working directory.

Notes

Some degree of validation is done on the file structure; however, it is generally unwise to mount corrupt file structures.

Be warned that when in single-user mode, the commands that look in */etc/mnttab* for default arguments (for example *df*, *ncheck*, *quot*, *mount*, and *umount*) give either incorrect results (due to a corrupt */etc/mnttab* from a non-*shutdown* stoppage) or no results (due to an empty *mnttab* from a *shutdown* stoppage).

If you are on a multi-user system this is not a problem; */etc/rc* initializes */etc/mnttab* to contain only */dev/root*; subsequent mounts update it appropriately.

The *mount* and *umount* commands use a lock file to guarantee exclusive access to */etc/mnttab*; the commands which just read it do not, so it is possible that they may hit a window when it is corrupt. This is not a problem in practice since *mount* and *umount* are not frequent operations.

Name

`mv`, `mvdir` – Moves or renames files/directories.

Syntax

`mv file1 file2`

`mv files ... directory`

`mvdir directory1 directory2`

Description

Mv moves (changes the name of) *file1* to *file2*.

If *file2* already exists, it is removed before *file1* is moved. If *file2* has a mode which forbids writing, *mv* prints the mode (see *chmod*(S)) and reads the standard input to obtain a line; if the line begins with ‘y’, the move takes place; if not, *mv* exits.

In the second form, one or more *files* are moved to the *directory* with their original filenames.

The *mvdir* command moves (changes the path of) *directory1* to *directory2*. For example, the command

```
mvdir oldir /a/b/d
```

renames the directory *oldir* to */a/b/c/oldir*. Both directories must already exist.

The *mvdir* command can only be run by a super-user.

Files

```
/bin/mv  
/etc/mvdir
```

See Also

chmod(S), *copy*(C), *cp*(C)

Notes

Mv refuses to move a file onto itself.

If *file1* and *file2* lie on different file systems, *mv* must copy the file and delete the original. In this case the owner name becomes that of the copying process and any linking relationship with other files is lost.

Name

ncheck – Generates names from inode numbers.

Syntax

ncheck [*-i numbers*] [*-a*] [*filesystem*]

Description

Ncheck with no argument generates a pathname vs. inode number list of all files on the set of file systems specified in */etc/mnttab*. The two characters, “/.”, are appended to the names of directory files. The *-i* option reduces the report to only those files whose inode numbers follow. The *-a* option allows printing of the names . and .., which are ordinarily suppressed. A single *filesystem* may be specified rather than the default list of mounted file systems.

Files

/etc/mnttab

See Also

fsck(C), *sort(C)*

Diagnostics

When the file system structure is improper, ?? denotes the “parent” of a parentless file and a pathname beginning with ... denotes a loop.

Notes

See *Notes* under *mount(C)*.



Name

netutil – Administers the XENIX network.

Syntax

netutil [*argument*] [*options*]

Description

The *netutil* command allows the user to create and maintain a network of XENIX machines. A Micnet network is a link through serial lines of two or more XENIX systems. It is used to send mail between systems with the *mail*(C) command, transfer files between systems with the *rcp*(C) command, and execute commands from a remote system with the *remote*(C) command.

The *netutil* command is used to create and distribute the data files needed to implement the network. It is also used to start and stop the network. The *argument* may be any one of *install*, *save*, *restore*, *start*, *stop*, or the numbers 1 through 5 respectively.

The *install* argument interactively creates the data files needed to run the network. The *save* argument saves these files on floppy disks, allowing them to be distributed to the other systems in the network. The *restore* argument copies the data files from floppy disk back to a system. The *start* argument starts the network. The *stop* argument stops the network. If invoked without an *argument*, the *netutil* command displays a menu from which to choose one. Once an *argument* is selected, it prompts for additional information if needed.

A network must be installed before it can be started. Installation consists of creating appropriate configuration files with the *install* argument. This argument requires the name of each machine in the network, the serial lines to be used to connect the machines, the speed of transmission for each line, and the names of the users on each machine. Once created, the files must be distributed to each computer in the network with the *save* and *restore* arguments.

The network is started by using the *start* argument on each machine in the network. When the network is started using the command “*netutil start*,” the system prompts with questions about logging errors and transactions. To suppress these prompts, the *start* argument must be followed by one of the following options:

- Suppresses prompts and does not create a log.
- e Suppresses prompts and logs all errors.
- x Suppresses prompts and logs all transactions.

If the “*netutil start*” command line is included in one of the files that are automatically invoked at startup, it must be followed by one of the above options or the system will hang waiting for a reply to a prompt. Once started, *mail* and *remote* commands can be passed along the network. A record of the transmissions between computers in a network can be kept in the network log files. Installation of the

network is described in the *XENIX Operations Guide*.

Files

/etc/netutil

Notes

The *netutil* command considers valid users as only those that have been specified as machine users in the network setup procedure that took place before aliases were set up. A user that happens to be valid for your current machine (i.e. has an entry in */etc/passwd* and a directory) will not be considered valid by *netutil* unless you have designated that user as such by using *netutil*.

See Also

aliases(M), aliashash(M), mail(C), micnet(M), rcp(C), remote(C), systemid(M), top(M)

XENIX Operations Guide

Name

newform – Changes the format of a text file.

Syntax

```
newform [-s] [-i tabspec] [-o tabspec] [-b n] [-e n] [-p n] [-a n] [-f] [-c char]
[-ln]
[file...]
```

Description

Newform reads lines from the named *files*, or the standard input if no input file is named, and reproduces the lines on the standard output. Lines are reformatted in accordance with command line options in effect.

Except for **-s**, command line options may appear in any order, may be repeated, and may be intermingled with *files*. Command line options are processed in the order typed. This means that option sequences like “**-e15 -l60**” will yield results different from “**-l60 -e15**”. Options are applied to all *files* on the command line.

- i***tabspec* Input tab specification: expands TABs to spaces, according to the tab specifications given. *Tabspec* recognizes all tab specification forms described in *tabs(C)*. In addition, *tabspec* may be —, in which *newform* assumes that the tab specification is to be found in the first line read from the standard input (see *fspec(C)*). If no *tabspec* is given, *tabspec* defaults to **-8**. A *tabspec* of **-0** expects no TABs; if any are found, they are treated as **-1**.
- o***tabspec* Output tab specification: replaces spaces by TABs, according to the tab specifications given. The tab specifications are the same as for **-i***tabspec*. If no *tabspec* is given, *tabspec* defaults to **-8**. A *tabspec* of **-0** means that no spaces will be converted to TABs on output.
- ln** Sets the effective line length to *n* characters. If *n* is not typed, **-l** defaults to 72 characters. The default line length without the **-l** option is 80 characters. Note that TABs and BACKSPACES are considered to be one character (use **-i** to expand TABs to spaces).
- bn** Truncates *n* characters from the beginning of the line when the line length is greater than the effective line length (see **-ln**). The default is to truncate the number of characters necessary to obtain the effective line length. The default value is used when **-b** with no *n* is used. This option can be used to delete the sequence numbers from a COBOL program as follows:

```
newform -l1 -b7 filename
```

The **-l1** option must be used to set the effective line length shorter than any existing line in the file so that the **-b** option is activated.

- en** Truncates *n* characters from the end of the line when the line length is greater than the effective line length (see **-ln**). Also see **-b**.
- ck** Changes the prefix/append character to *k*. The default character for *k* is a SPACEBAR (see options **-p** and **-c**).
- pn** Prefixes *n* characters (see **-ck**) to the beginning of a line when the line length is less than the effective line length. The default is to prefix the number of characters necessary to obtain the effective line length.
- an** Appends *n* characters to the end of a line. The default is to append the number of characters necessary to get the effective line length.
- f** Writes the tab specification format line on the standard output before any other lines are output. The tab specification format line which is printed will correspond to the format specified in the *last -o* option. If no **-o** option is specified, the line which is printed will contain the default specification of **-8**.
- s** Shears off leading characters on each line up to the first TAB and places up to eight of the sheared characters at the end of the line. If more than eight characters (not counting the first TAB) are sheared, the eighth character is replaced by a * and any characters to the right of it are discarded. The first TAB is always discarded.

An error message and program exit will occur if this option is used on a file without a TAB on each line. The characters sheared off are saved internally until all other options specified are applied to that line. The characters are then added at the end of the processed line.

Example

In the following example, *newform* converts a file named *text* with leading digits, one or more TABs, and text on each line to a file beginning with the text and the leading digits placed at the end of each line in column 73 (**-s** option). All TABs after the first one are expanded to spaces (**-i** option). To reach the line length of 72 characters (**-l** option), spaces are appended to each line up to column 72 (**-a** option) or lines are truncated at column 72 (**-e** option). To reformat the sample file, *text*, in this manner, type:

```
newform -s -i -l -a -e text
```